



UNIT V - Stack

Basic concept

- Stack is a linear data structure in which the insertion and deletion operations are performed at only one end.
- In a stack, adding and removing of elements are performed at a single position which is known as "**top**".
- That means, a new element is added at top of the stack and an element is removed from the top of the stack.
- In stack, the insertion and deletion operations are performed based on **LIFO** (Last In First Out) principle.
- The data structure stack is very similar to these practical examples



Stack of books

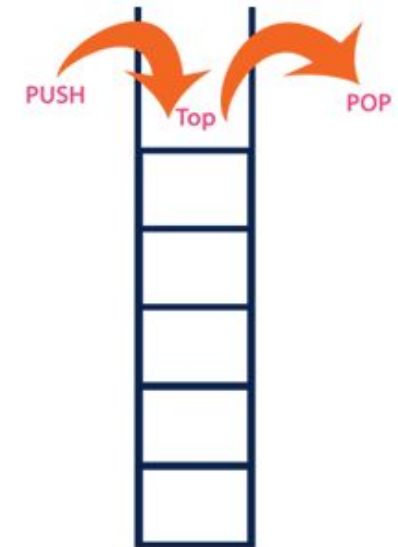


Stack of chairs



Stack of cups

- In a stack, the insertion operation is performed using a function called "push" and deletion operation is performed using a function called "pop".
- In the figure, PUSH and POP operations are performed at a top position in the stack. That means, both the insertion and deletion operations are performed at one end (i.e., at Top)
- A stack data structure can be defined as follows...
- ***Stack is a linear data structure in which the operations are performed based on LIFO principle.***
- Stack can also be defined as
- ***"A Collection of similar data items in which both insertion and deletion operations are performed based on LIFO principle".***



Stack Abstract Data Type

- A stack is defined as a restricted list where all insertions and deletions are made only at one end, the *top*.
- Each stack abstract data type (ADT) has a data member, commonly named as *top*, which points to the topmost element in the stack.
- There are two basic operations *push* and *pop* that can be performed on a stack; insertion of an element in the stack is called *push* and deletion of an element from the stack is called *pop*.
- In stacks, we cannot access data elements from any intermediate positions other than the top position.

Primitive Operations

1. **Push**—inserts an element on the top of the stack
2. **Pop**—deletes an element from the top of the stack
3. **GetTop**—reads (only reading, not deleting) an element from the top of the stack
4. **Stack_initialization**—sets up the stack in an empty condition
5. **Empty**—checks whether the stack is empty
6. **Full**—checks whether the stack is full

ADT Stack(element)

1. Declare Create() $\rightarrow$ stack
2. push(element, stack) $\rightarrow$ stack
3. pop(stack) $\rightarrow$ stack
4. getTop(stack) $\rightarrow$ element
5. Is_Empty(stack) $\rightarrow$ Boolean;
6. for all $S \in \text{stack}$, $e \in \text{element}$, Let
7. Is_Empty(Create) = true
8. Is_Empty(push(e, S)) = false
9. pop(Create()) = error
10. pop(push(e, S)) = S
11. getTop(Create) = error
12. getTop(push(e, S)) = e
13. end
14. end stack

Push

- The push operation inserts an element on the top of the stack.
- The recently added element is always at the top of the stack.
- Before every push, we must ensure whether there is a room for a new element

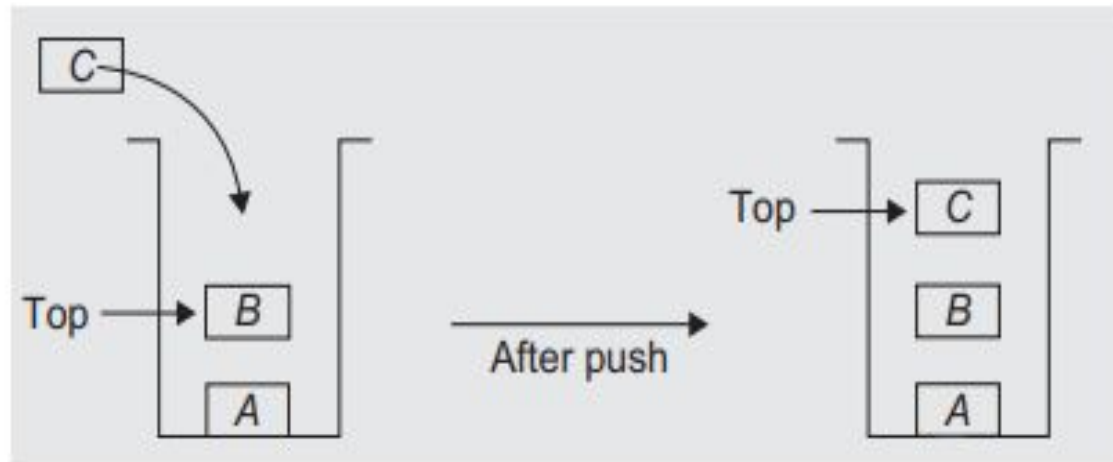


Fig. 3.3 The push operation

- When there is no space to accommodate the new element on the stack, the stack is said to be full.
- If the operation push is performed when the stack is full, it is said to be in overflow state, that is, no element can be added when the stack is full.
- The push operation modifies the top since the newly inserted element becomes the topmost element

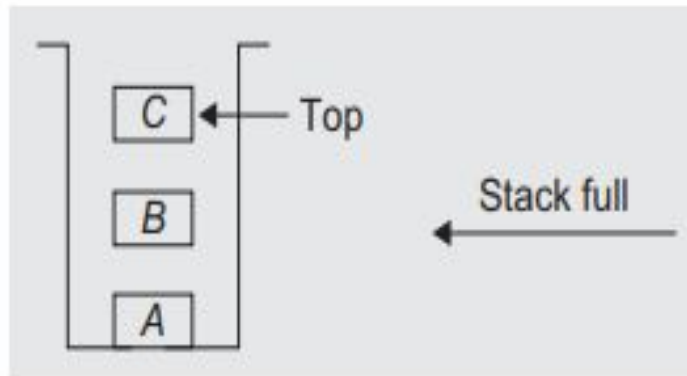


Fig. 3.4 The stack full condition (stack capacity = 3)

Pop

- The pop operation deletes an element from the top of the stack and returns the same to the user.
- It modifies the stack so that the next element becomes the top element



Fig. 3.5 The pop operation

- When there is no element available on the stack, the stack is said to be empty.
- If pop is performed when the stack is empty, then the stack is said to be in an underflow state .
- The pop operation should not be performed when the stack is empty, and hence before every pop, we must ensure that the stack is not empty.
- After deleting the last element from the stack, the stack should be set to an empty state.

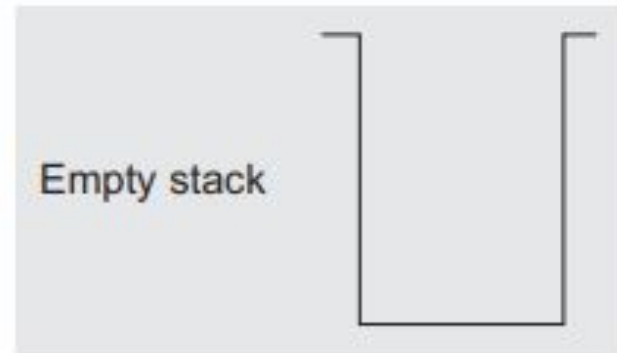


Fig. 3.6 The empty stack

GetTop

- The getTop operation gives information about the topmost element and returns the element on the top of the stack.
- In this operation, only a copy of the element, which is at the top of the stack, is returned.
- Hence, the top is still set to the same element, this is the key difference between the pop and getTop operations.
- The getTop operation does not modify the variable top.
- It signals the stack underflow error if the stack is empty.

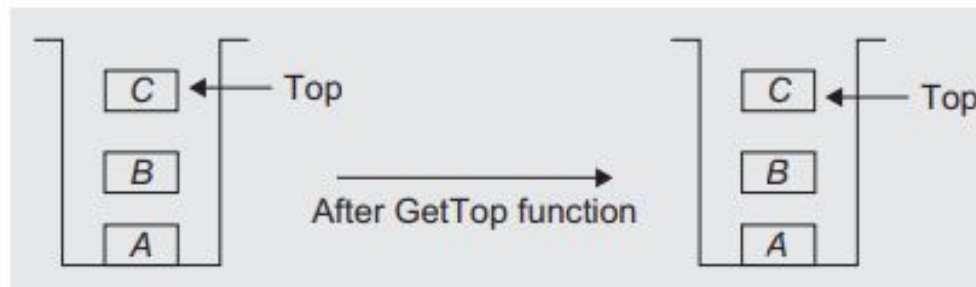


Fig. 3.7 The getTop operation

- As both insert and delete operations are allowed only at one end of the stack, it retrieves data in the reverse order in which the data is stored.
- In Fig , let $S = \{A, B, C\}$.
- Suppose that the order of the operations is push(A), push(B), and then push(C).
- When we remove these elements out of the stack, they will be removed in the order C, B, and then A.

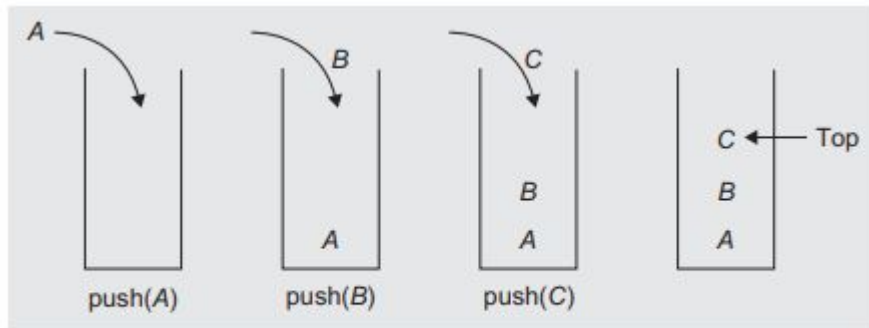


Fig. 3.8 Stack and push operations

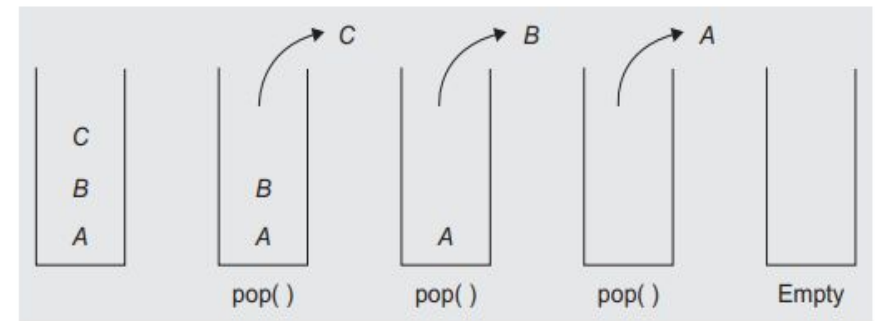


Fig. 3.9 Stack and pop operations

Representation of Stacks using Sequential Organization(Arrays)

- A stack can be implemented using both a static data structure (array) and a dynamic data structure (linked list).
- The simplest way to represent a stack is by using a one-dimensional array.
- A stack implemented using an array is also called a contiguous stack.
- An array is used to store an ordered list of elements.
- A stack is an ordered collection of elements.
- Hence, it would be very simple to manage a stack when represented using an array.
The only difficulty with an array is its static memory allocation.
- This is because we declare an array to be of arbitrarily maximum size before compilation.

Figure shows the realization of a stack using arrays.

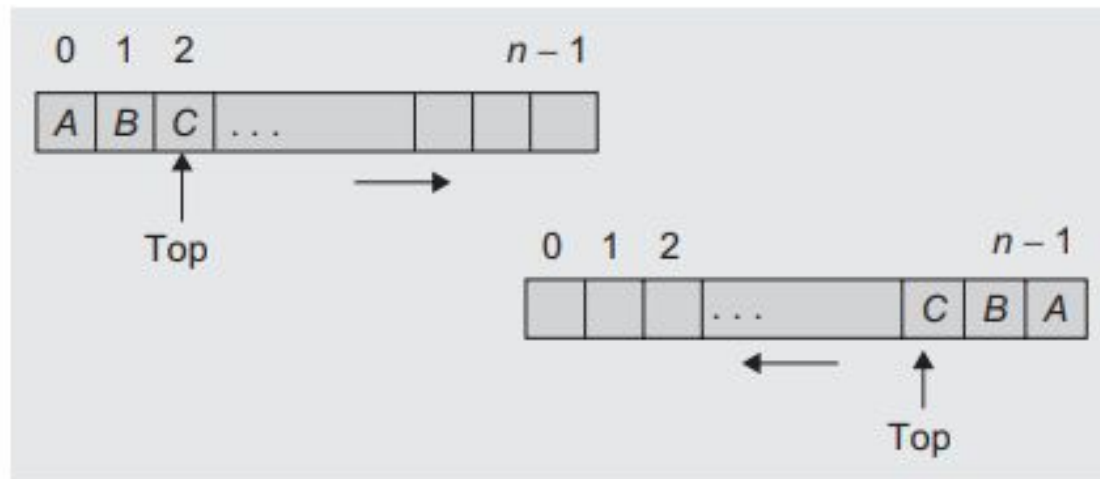


Fig. 3.10 Stack using array

Create

- The stack when created is initially empty.
- The implementation of the stack could be using an array or using a linked list implementation.
- For array implementation, its size should be predefined, and its implementation time should not exceed run-time.

```
int Stack[100];  
int top = -1;
```

- These statements create an empty stack of size 100, which will hold integer values, and the variable top is initialized to -1.

Empty

- Empty is an operation that takes the stack as an argument, checks whether it is empty or not, and returns the Boolean value true or false, respectively.
- The stack empty state can be checked by comparing the value of top with the value **-1**, because **top = -1** represents an empty stack.

```
if(top == -1)
```

```
    return 1;
```

```
else
```

```
    return 0;
```


GetTop

- The getTop operation checks for the stack empty state.
- If the stack is empty, it reports the 'stack underflow' error message; else it returns a copy of the element that is at the top of the stack.
- Here, top is not updated as the element is not deleted from the stack; rather, the element is still at the top location. The element is just read from the stack.
- Hence, its behavior can be described using the following statement:

```
if(top == -1)

    cout << "Stack underflow (empty)" << endl;

else

    return(Stack[top]);
```

Push

- The push operation inserts an element onto the stack of maximum size MaxCapacity.
- Element insertion is possible only if the stack is not full.
- The stack is practically full when the array size exceeds.
- Hence, the stack full state can be verified by comparing the top with MaxCapacity - 1.
- If the stack is not full, the top is incremented by 1 and the element is added on the top of the stack. In brief,

```
if(top == MaxCapacity - 1)
    cout << "Stack overflow (full)";
else
{
    top ++; //increment top by one
    stack[top] = Element; //add the element in new top position
}
```

Pop

- The pop operation deletes the element at the top of the stack and returns the same.
- This is done only if the stack is not empty. If the stack is empty, no deletion is possible. This is checked by the empty() function.
- If the stack is not empty, then the element at the top of the stack is returned and the top is decreased by one.
- This is executed as

```
if(top == -1)
    cout << "Stack underflow\n";
else
    return(Stack[top--]);
```

```
#include <iostream>
using namespace std;
#define SIZE 100
void push(int);
void pop();
void display();
int stack[SIZE], top = -1;
int main() {
    int value, choice;
    while (1) {
        cout << "\n\n***** MENU *****" << endl;
        cout << "1. Push\n2. Pop\n3. Display\n4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;
```

```
switch (choice) {
    case 1:
        cout << "Enter the value to be inserted: ";
        cin >> value;
        push(value);
        break;
    case 2:
        pop();
        break;
    case 3:
        display();
        break;
    case 4:
        return 0;
    default:
        cout << "\n Wrong selection!!! Try again!!!" << endl;
}
}
return 0;
}
```

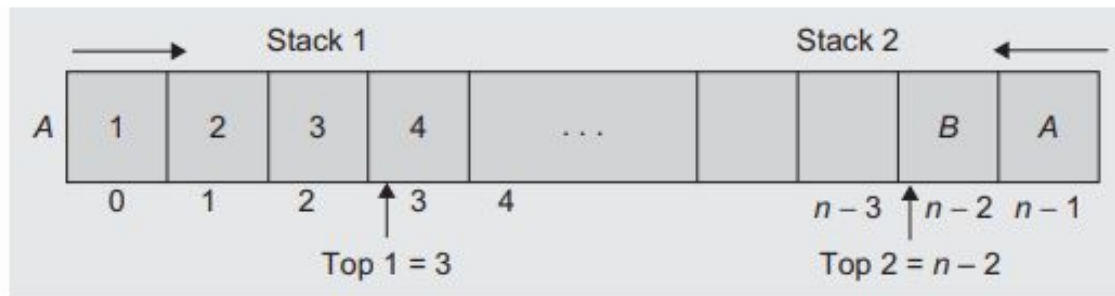
```
void push(int value)
{
    if (top == SIZE - 1)
        cout << "\n Stack is Full!!! Insertion is not possible!!!" << endl;
    else {
        top++;
        stack[top] = value;
        cout << "\n Insertion success!!!" << endl;
    }
}
```

```
void pop() {
    if (top == -1)
        cout << "\n Stack is Empty!!! Deletion is not possible!!!" << endl;
    else {
        cout << "\n Deleted : " << stack[top] << endl;
        top--;
    }
}
```

```
void display()
{
    if (top == -1)
        cout << "\nStack is Empty!!!" << endl;
    else {
        cout << "Stack elements are:" << endl;
        for (int i = top; i >= 0; i--)
            cout << stack[i] << endl;
    }
}
```

Multiple Stacks

- Often, data is represented using several stacks. The contiguous stack (stack using an array) uses separate arrays for more than one stack, if needed.
- The use of a contiguous stack when more than one stack is needed is not a space-efficient approach, because many locations in the stacks are often left unused.
- An efficient solution to this problem is to use a single array to store more than one stack. Figure shows two stacks using one array.



- Multiple stacks can be implemented by sequentially mapping these stacks into $A[0]$, ..., $A[n-1]$.
- The solution is simple if we implement only two stacks. The first stack grows towards $A[n-1]$ from $A[0]$ and the second stack grows towards $A[0]$ from $A[n-1]$.

APPLICATIONS of stack

The stack data structure is used in a wide range of applications. A few of them are the following:

1. Converting infix expression to postfix and prefix expressions
2. Evaluating the postfix expression
3. Checking well-formed (nested) parenthesis
4. Reversing a string
5. Processing function calls
6. Parsing (analyze the structure) of computer programs
7. Simulating recursion
8. In computations such as decimal to binary conversion
9. In backtracking algorithms (often used in optimizations and in games)

EXPRESSION EVALUATION AND CONVERSION

- The most frequent application of stacks is in the evaluation of arithmetic expressions.
- An arithmetic expression is made of operands, operators, and delimiters.
- A complex assignment statement such as
- The first problem in understanding the meaning $X = (A/B + C \times D - F \times G/Q)$ is to decide the order in which the operations are to be carried out. This demands that every language must uniquely define such an order.

Arithmetic, boolean, and relational operators	Priority
$\wedge$, Unary $+$, Unary $-$, $\sim$	1
$\times$, $/$	2
$+$, $-$	3
$<$, $\leq$, $=$, $\neq$, $\geq$, $>$	4
AND	5
OR	6

Polish Notation and Expression Conversion

- Polish notation, which gives two alternatives to represent an arithmetic expression, namely the postfix and prefix notations.
- The fundamental property of Polish notation is that the order in which the operations are to be performed is determined by the positions of the operators and operands in the expression.
- Hence, the advantage is that parentheses is not required while writing expressions in Polish notation.
- The conventional way of writing the expression is called infix, because the binary operators occur between the operands, and unary operators precede their operand.
- For example, the expression $((A + B) \times C)/D$ is an infix expression.
- In postfix notation, the operator is written after its operands, whereas in prefix notation, the operator precedes its operands.

Example

Consider the following Infix Expression to be converted into Postfix Expression...

$$D = A + B * C$$

Step 1 - The Operators in the given Infix Expression : = , + , *

Step 2 - The Order of Operators according to their preference : * , + , =

Step 3 - Now, convert the first operator * ----- $D = A + B C *$

Step 4 - Convert the next operator + ----- $D = A B C * +$

Step 5 - Convert the next operator = ----- $D A B C * + =$

Finally, given Infix Expression is converted into Postfix Expression as follows...

$$D A B C * + =$$

- Table shows one sample expression in all three notations.

Infix	Prefix	Postfix
(operand)(operator)(operand)	(operator)(operand)(operand)	(operand)(operand)(operator)
$(A + B) \times C$	$\times + ABC$	$AB + C \times$

- In Example , the conversion of an expression to its postfix and prefix notations is discussed.
- Example Convert the following expression to its postfix and prefix notations:

$$X = A/B \wedge C + D \times E - A \times C$$

Solution By applying the rules of priority and associativity, this expression can be written in the following form:

$$X = ((A/(B \wedge C)) + (D \times E) - (A \times C))$$

It can be reworked to get its equivalent postfix and prefix expressions.

$$\text{Postfix: } ABC \wedge / DE \times + AC \times -$$

$$\text{Prefix: } - + / A \wedge BC \times DE \times AC$$

Need for Prefix and Postfix Expressions

- Evaluation of an infix expression using a computer needs proper code generation by the compiler without any ambiguity and is difficult because of various aspects such as the operator's priority and associativity.
- This problem can be overcome by writing or converting the infix expression to an alternate notation such as the prefix or the postfix. The postfix and prefix expressions possess many advantages as follows:
 1. The need for parenthesis as in an infix expression is overcome in postfix and prefix notations.
 2. The priority of operators is no longer relevant.
 3. The order of evaluation depends on the position of the operator but not on priority and associativity.
 4. The expression evaluation process is much simpler than attempting a direct evaluation from the infix notation.

Postfix Expression Evaluation

- The postfix expression may be evaluated by making a left-to-right scan, stacking operands, and evaluating operators using the correct number from the stack as operands and again placing the result onto the stack.
- This evaluation process is much simpler than attempting a direct evaluation from the infix notation.
- This process continues till the stack is not empty or on occurrence of the character #, which denotes the end of the expression

Algorithm 3.1 lists the steps involved in the evaluation of the postfix expression E .

ALGORITHM 3.1

1. Let E denote the postfix expression
 2. Let Stack denote the stack data structure to be used & let Top = -1
 3. while(1) do
begin
 $X = \text{get_next_token}(E)$ // Token is an operator, operand, or delimiter
 if($X = \#$) {end of expression}
 then return
 if(X is an operand)
 then push(X) onto Stack
 else { X is operator}
 begin
 $OP1 = \text{pop}()$ from Stack
 $OP2 = \text{pop}()$ from Stack
 $\text{Tmp} = \text{evaluate}(OP1, X, OP2)$
 push(Tmp) on Stack
 end
 {If X is operator then pop the correct number of operands from stack for operator X . Perform the operation and push the result, if any, onto the stack}
 end
end
 4. stop
-

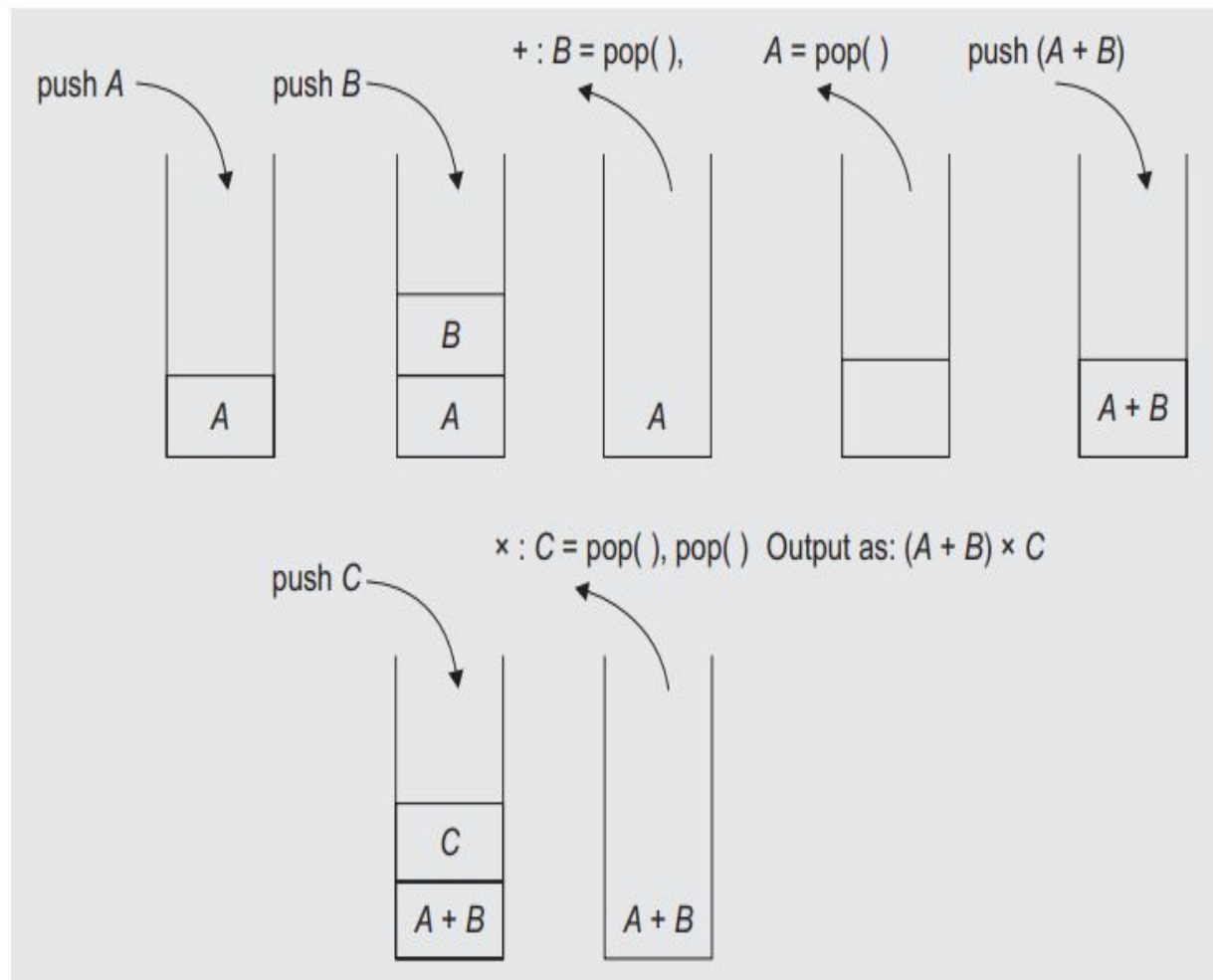


Fig. 3.13 Evaluation of postfix expression $AB + C \times$

An expression in one form can be converted to the other two forms. Let us write algorithms for all these conversions.

1. Infix expression to postfix expression
2. Infix expression to prefix expression
3. Prefix expression to infix expression
4. Prefix expression to postfix expression
5. Postfix expression to infix expression
6. Postfix expression to prefix expression

Infix to Postfix Conversion using Stack Data Structure

To convert Infix Expression into Postfix Expression using a stack data structure, We can use the following steps...

1. Read all the symbols one by one from left to right in the given Infix Expression.
2. If the reading symbol is **operand**, then directly print it to the result (Output).
3. If the reading symbol is **left parenthesis '('**, then Push it on to the Stack.
4. If the reading symbol is **right parenthesis ')'**, then Pop all the contents of stack until respective left parenthesis is popped and print each popped symbol to the result.
5. If the reading symbol is **operator (+ , - , * , / etc.,)**, then Push it on to the Stack. However, first pop the operators which are already on the stack that have higher or equal precedence than current operator and print them to the result.

Convert an infix expression to its postfix form.

- Let us consider an example $E = A/B \wedge C + D \times E - A \times C$.
- Let us fully parenthesize the same as $E = (((A/(B \wedge C)) + (D \times E)) - (A \times C))$
- Let us move all operators to the corresponding right parenthesis and replace the same.

$$E = (((A/(B \wedge C)) + (D \times E)) - (A \times C))$$

Now let us eliminate all parentheses. We get the postfix equivalent of the infix expression.

$$E(\text{postfix}) = ABC \wedge / DE \times + AC \times -$$

This method can be used to get an equivalent prefix notation too as follows:

$$(((A/(B \wedge C)) + (D \times E)) - (A \times C))$$

- We now get the prefix expression after eliminating the parentheses as

$$E(\text{prefix}) = - + / A \wedge B C \times D E \times A C$$

- Let us try to work out the algorithm to convert an infix to a postfix (also to prefix).

Table 3.3 Infix to postfix conversion of the expression $E = A + B \times C$

Next character	Stack	Output
A	Empty	A
$+$	$+$	A
B	$+$	AB

Table 3.4 Handling and stacking of the $\times$ operator in expressions

(a) Handling of the $\times$ operator		
	Infix	Postfix
Examples	$X_1 = (A + B) \times C$ $X_2 = A + (B \times C)$	$AB + C \times$ $ABC \times +$

Table 3.4 (Continued)

(b) Stacking of the $\times$ operator		
Next character	Stack	Output
A	Empty	A
$+$	$+$	A
B	$+$	AB
$\times$	$+\times$	AB
C	$+\times$	ABC
# (Pop all)	$+\times$	$ABC \times +$

Table 3.5 Infix to postfix conversion of the expression $A \times (B + C) \times D$

Next character	Stack	Output
A	Empty	A
$\times$	$\times$	A
(	$\times($	A
B	$\times($	AB
+	$\times(+$	AB
C	$\times(+$	ABC

Table 3.5 (Continued)

Next character	Stack	Output
)	$\times$	ABC+
$\times$	$\times$	ABC+ $\times$
D	$\times$	ABC+ $\times$ D
Done	Empty	ABC+ $\times$ D $\times$

Table 3.6 Infix to postfix conversion of the expression $E = A \times B + C$

Next character	Stack	Output
A	Empty	A
$\times$	$\times$	A
B	$\times$	AB
+	+	AB $\times$
C	+	AB $\times$ C
# (Pop all)	+	AB $\times$ C+

Table 3.8 The operator and its ISP and ICP

Symbol	In-stack priority (ISP)	Incoming priority (ICP)
)	—	—
$\wedge$	3	4
$\times/$	2	2
$+-$	1	1
(	0	4

Infix to Postfix Conversion

ALGORITHM 3.2

1. Scan expression E from left to right, character by character, till character is '#'
 `ch = get_next_token(E)`
 2. `while(ch != '#')`
 - `if(ch = '(')` then `ch = pop()`
 `while(ch != '(')`
 - Display ch
 - `ch = pop()`
 - end while
 - `if(ch = operand)` display the same
 - `if(ch = operator)` then
 - `if(ICP > ISP)` then `push(ch)`
 - else
 - `while(ICP <= ISP)`
 - pop the operator and display it
 - end while
 - `ch = get_next_token(E)`
 - end while
 3. `if(ch = #)` then `while(!emptystack())` pop and display
 4. stop
-

EXAMPLE 3.2

Convert the following infix expression to its postfix form:

$$A \wedge B \times C - C + D/A/(E + F)$$

Solution Conversion of infix to postfix form can be illustrated as in Table 3.9**Table 3.9** Infix to postfix conversion of the expression $A \wedge B \times C - C + D/A/(E + F)$

Character scanned	Stack contents	Postfix expression
A	Empty	A
^	^	A
B	^	AB
×	×	AB^
C	×	AB ^ C
-	-	AB ^ C ×
C	-	AB ^ C × C
+	+	AB ^ C × C -
D	+	AB ^ C × C - D
/	+/	AB ^ C × C - D
A	+/	AB ^ C × C - DA
/	+/	AB ^ C × C - DA/
(	+/(	AB ^ C × C - DA/
E	+/(	AB ^ C × C - DA/E
+	+/(+	AB ^ C × C - DA/E
F	+/(+	AB ^ C × C - DA/EF
)	+/	AB ^ C × C - DA/EF+
	Empty	AB ^ C × C - DA/EF+/+

Infix to Prefix Conversion

- For converting the infix expression to a prefix expression, two stacks are needed **the operator Stack** and **the display Stack**.
- The display Stack stores the prefix expression

ALGORITHM 3.3

```
1. Scan expression E, character by character from right to left
   ch = get_next_token(E)
2. while(ch != '#') do
   if(ch = operand) then push(ch) in display Stack
   if (ch = ')') then
     ch = pop() from operator Stack
   while(ch != '(')
     push(ch) in display Stack
     ch = pop()
   end while
   if(ch = operator) then
     if ICP(op) >= ISP(op) then
       push ch in operator Stack
     else
       ch = pop()
       while(ICP < ISP)
```

```
        ch = pop() from operator Stack and push 'ch' in
        display Stack
    end while
    ch = get_next_token(E)
end while
3. if (ch = '#') then
    while(!emptystack(operator))
        ch = pop(operator)
        push ch on display stack
    end while
4. while(!emptystack(display))
    ch = pop(operator)
    display ch
end while
5. stop
```

Example 3.3 illustrates the conversion of an infix expression to its prefix form.

EXAMPLE 3.3 Convert the following infix expression to its corresponding prefix form:

$$A \wedge B \times C - C + D/A/(E + F)$$

Solution The conversion to prefix notation is as given in Table 3.10

Table 3.10 Infix to prefix conversion of the expression $A \wedge B \times C - C + D/A/(E + F)$

Character scanned	Stack	Prefix expression
)	)	
F	)	F
+	) +	F
E	) +	EF
(	Empty	+EF
/	/	+EF
A	/	A + EF
/	//	A + EF
D	//	DA + EF
+	+	//DA + EF
C	+	C//DA + EF
-	+ -	C//DA + EF
C	+ -	CC//DA + EF
×	+ - ×	CC//DA + EF
B	+ - ×	BCC//DA + EF
^	+ - × ^	BCC//DA + EF
A	+ - × ^	ABCC//DA + EF
	Empty	+ - × ^ ABCC//DA + EF

Postfix to Infix Conversion

ALGORITHM 3.4

1. Scan expression E from left to right character by character
 `ch = get_next_token(E)`
 2. `while(ch != '#')` do
 `if(ch = operand) then push(ch)`
 `if(ch = operator) then`
 begin
 `t2 = pop() and t1 = pop()`
 `push(strcat['(', t1, ch, t2, ''])`
 end
 `ch = get_next_token(E)`
end while
 3. `if ch = '#', while(!emptystack()) pop and display`
 4. stop
-

Example 3.4 illustrates the conversion of a postfix expression to its infix form.

EXAMPLE 3.4 Convert the following postfix expression to its infix form:

$$AB \wedge C \times C - DA/EE +/+$$

Solution The conversion of the given postfix expression to its infix form is given in Table 3.11.

Table 3.11 Postfix to infix conversion of the expression $AB \wedge C \times C - DA/EE+/+$

Character scanned	Stack contents
A	A
B	AB
^	$A \wedge B$
C	$A \wedge B, C$
×	$A \wedge B \times C$
C	$A \wedge B \times C, C$
-	$A \wedge B \times C - C, D$
D	$A \wedge B \times C - C, D$
A	$A \wedge B \times C - C, D, A$
/	$A \wedge B \times C - C, D/A$
E	$A \wedge B \times C - C, D/A, E$
E	$A \wedge B \times C - C, D/A, E, E$
+	$A \wedge B \times C - C, D/A, E + E$
/	$A \wedge B \times C - C, D/A/E + E$
+	$A \wedge B \times C - C + D/A/E + E$

Postfix to Prefix Conversion

ALGORITHM 3.5

1. Scan expression E from left to right character by character
 `ch = get_next_token(E)`
 2. `while(ch != '#')` do
 `if(ch = operand) then push(ch)`
 `if(ch = operator) then`
 begin
 `t2 = pop() and t1 = pop()`
 `push(strcat(ch, t1, t2)`
 end
 `ch = get_next_token(E)`
 end while
 3. `if ch = '#', while(!emptystack()) pop and display`
 4. stop
-

EXAMPLE 3.5

Convert the following postfix expression to its prefix form:

$$AB \wedge C \times C - DA/EE+ / +$$

Solution The conversion of the given postfix expression to its infix form is given in Table 3.12.

Table 3.12 Postfix to prefix conversion of the expression $AB \wedge C \times C - DA/EE+ / +$

Character scanned	Stack contents
A	A
B	AB
^	^AB
C	^ABC
×	×^ABC
C	×^ABC, C
−	−×^ABCC
D	−×^ABCC, D
A	−×^ABCC, D, A
/	−×^ABCC, /DA
E	−×^ABCC, /DA, E
E	−×^ABCC, /DA, E, E
+	−×^ABCC, /DA, +EE
/	−×^ABCC, //DA + EE
+	+−^ABCC//DA + EE

Prefix to Infix Conversion

ALGORITHM 3.6

1. Scan expression E from right to left character by character
 `ch = get_next_token(E)`
 2. `while(ch != '#')` do
 `if(ch = operand) then push(ch)`
 `if(ch = operator) then`
 begin
 `t2 = pop() and t1 = pop()`
 `push(strcat['(', t1, ch, t2, ''])`
 end
 `ch = get_next_token(E)`
end while
 3. `if ch = '#', while(!emptystack()) pop and display`
 4. stop
-

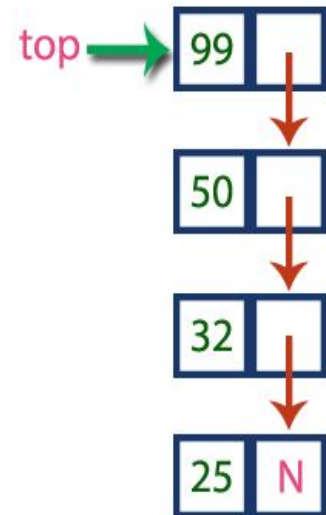
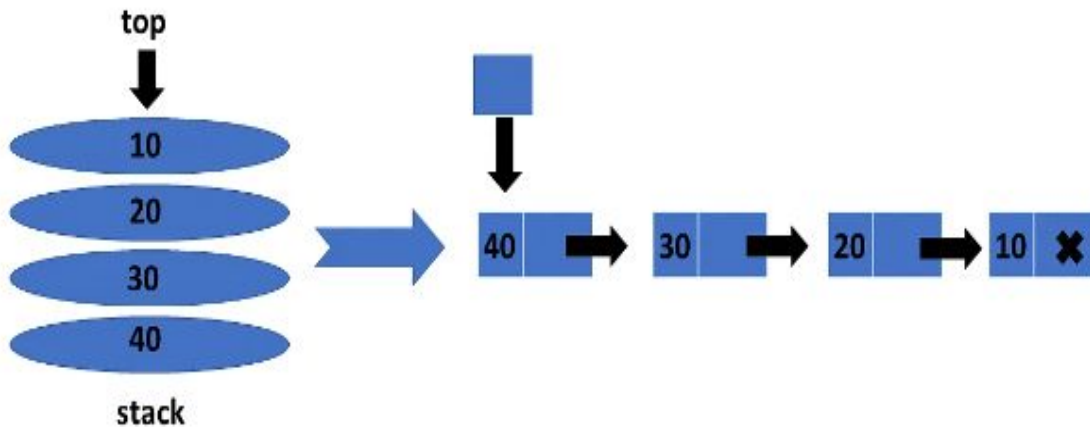
Prefix to Postfix Conversion

ALGORITHM 3.7

1. Scan expression E from left to right character by character
 `ch = get_next_token(E)`
 2. `while(ch != '#')` do
 `if(ch = operand) then push(ch)`
 `if(ch = operator) then`
 begin
 `t2 = pop() and t1 = pop()`
 `push(strcat [t1, t2, ch])`
 end
 `ch = get_next_token(E)`
 end while
 3. `if ch = '#', while(!emptystack()) pop and display`
 4. stop
-

Stack Using Linked List

- In linked list implementation of a stack, every new element is inserted as '**top**' element.
- That means every newly inserted element is pointed by '**top**'.
- Whenever we want to remove an element from the stack, simply remove the node which is pointed by '**top**' by moving '**top**' to its previous node in the list.
- The **next** field of the first element must be always **NULL**.



Stack Operations using Linked List

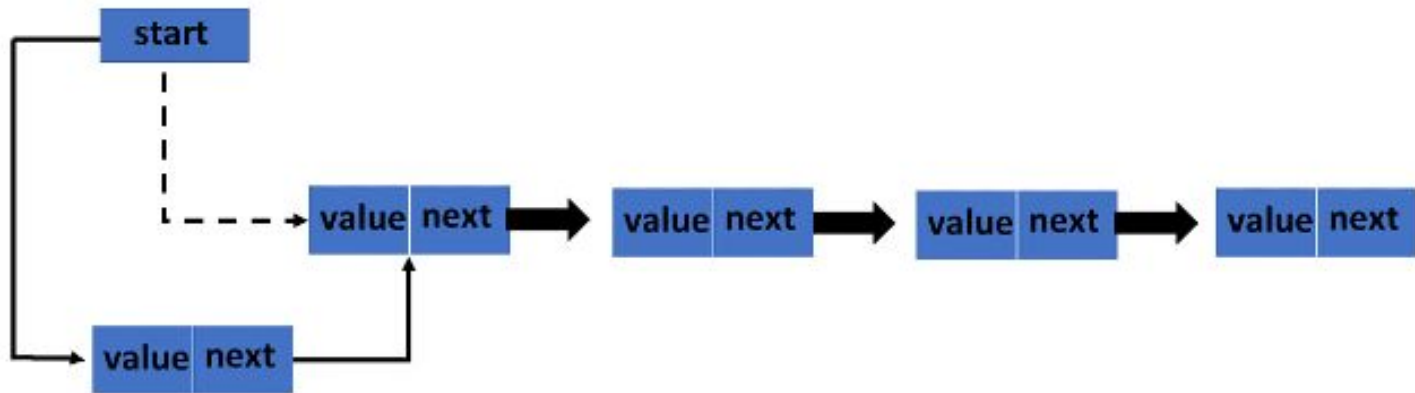
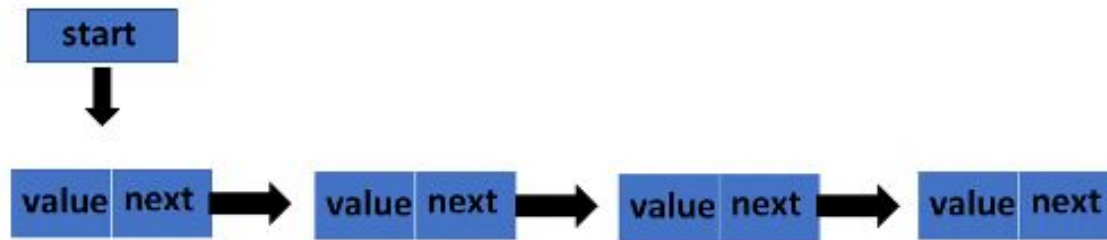
To implement a stack using a linked list, we need to set the following things before implementing actual operations.

- **Step 1** - Include all the **header files** which are used in the program. And declare all the **user defined functions**.
- **Step 2** - Define a '**Node**' structure with two members **data** and **next**.
- **Step 3** - Define a **Node** pointer '**top**' and set it to **NULL**.
- **Step 4** - Implement the **main** method by displaying Menu with list of operations and make suitable function calls in the **main** method.

push(value) - Inserting an element into the Stack

We can use the following steps to insert a new node into the stack...

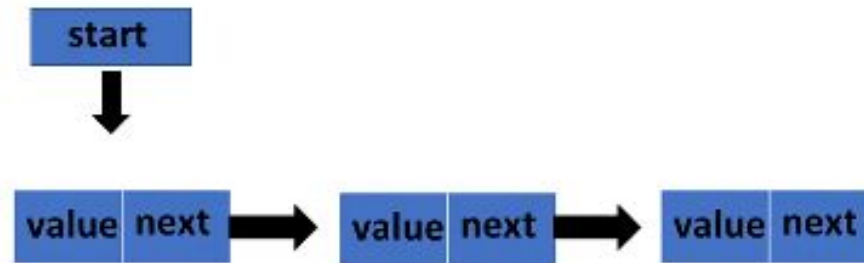
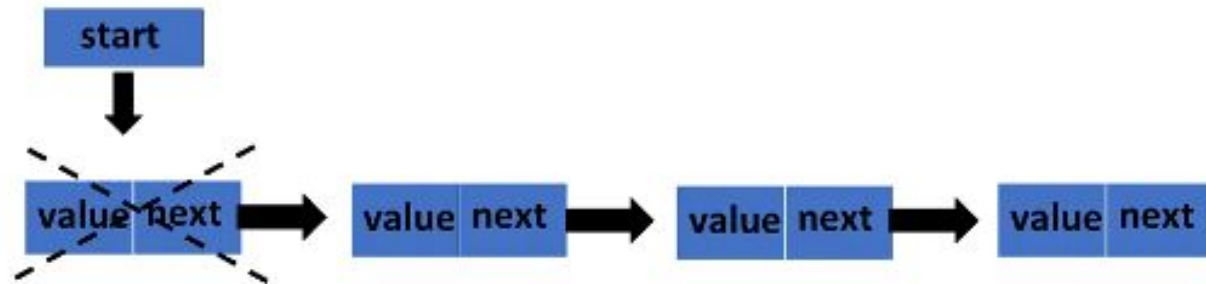
- **Step 1** - Create a **newNode** with given value.
- **Step 2** - Check whether stack is **Empty** (**top == NULL**)
- **Step 3** - If it is **Empty**, then set **newNode** $\rightarrow$ **next** = **NULL**.
- **Step 4** - If it is **Not Empty**, then set **newNode** $\rightarrow$ **next** = **top**.
- **Step 5** - Finally, set **top** = **newNode**.



pop() - Deleting an Element from a Stack

We can use the following steps to delete a node from the stack...

- **Step 1** - Check whether **stack** is **Empty** (**top == NULL**).
- **Step 2** - If it is **Empty**, then display "**Stack is Empty!!! Deletion is not possible!!!**" and terminate the function
- **Step 3** - If it is **Not Empty**, then define a **Node** pointer '**temp**' and set it to '**top**'.
- **Step 4** - Then set '**top = top → next**'.
- **Step 5** - Finally, delete '**temp**'. (**free(temp)**).



display() - Displaying stack of elements

We can use the following steps to display the elements (nodes) of a stack...

- **Step 1** - Check whether stack is **Empty** (**top == NULL**).
- **Step 2** - If it is **Empty**, then display '**Stack is Empty!!!**' and terminate the function.
- **Step 3** - If it is **Not Empty**, then define a Node pointer '**temp**' and initialize with **top**.
- **Step 4** - Display '**temp → data --->**' and move it to the next node. Repeat the same until **temp** reaches to the first node in the stack. (**temp → next != NULL**).
- **Step 5** - Finally! Display '**temp → data ---> NULL**'.

Recursion - concept

- When a function calls itself, either directly or indirectly, it is said to be making a recursive call.
- A program becomes compact and readable with recursive functions.
- Recursion is extremely powerful as it enables the programmer to express complex processes easily.
- Recursive programs are used in a variety of applications ranging from calculating the factorial of a number to playing complex games against human intelligence.

- Let us consider an example of computing the factorial of a number. Factorial is a mathematical term. The factorial of a number, say n , is equal to the product of all the integers from 1 to n .
- The factorial of n is denoted as

$$n! = 1 \times 2 \times 3 \times \dots \times n \text{ or } n! = n \times n - 1 \times \dots \times 1$$

- For example, $10! = 1 \times 2 \times 3 \times 4 \times 5 \times 6 \times 7 \times 8 \times 9 \times 10$.
- The simplest program to calculate the factorial of a number is by using a loop with a product variable.
- Algorithm states the iterative process of computing the factorial of n as
- $10! = 10 \times 9 \times 8 \times \dots \times 1$.

ALGORITHM 4.1

An iterative version of an algorithm to compute the factorial of a number

1. start
 2. Let n be the number whose factorial is to be computed and let
 Factorial = 1
 3. while($n > 1$) do
 begin
 Factorial = Factorial * n
 $n = n - 1$
 end
 4. stop
-

- This recursive definition of factorial has two steps, as follows:

1. If $n = 1$, then factorial of $n = 1$
2. Otherwise, factorial of $n = n \times \text{factorial of } (n - 1)$

PROGRAM CODE 4.1

```
int Factorial(int n)
{
    if(n == 1)          // end condition
        return 1;
    else
        return Factorial(n - 1) * n;
}
```

Variants of Recursion

- Depending on the following characterization, the recursive functions are categorized as direct, indirect, linear, tree, and tail recursions.
- Recursion may have any one of the following forms:
 1. A function calls itself.
 2. A function calls another function which in turn calls the caller function.
 3. The function call is part of the same processing instruction that makes a recursive function call.

Direct recursion –

- **Recursion** is when a function calls itself. Recursion is said to be direct when a function calls itself directly, and it is said to be indirect when it calls another function which in turn calls it. The Factorial() function we discussed in Program Code is an example of direct recursion. The Power() function in Program Code below is for computing the value of Equation

recursively
$$X^Y = X \times X^{Y-1}$$

PROGRAM CODE 4.6

```
int Power(int x, int y)
{
    if(y == 1)
        return x;
    else
        return (x * Power(x, y - 1));
}
```

Indirect recursion –

- A function is said to be indirectly recursive if it calls another function, which in turn calls it. Program Code is an example of an indirect recursion, where the function Fact() calls the function Dummy(), and the function Dummy() in turn calls Fact().

PROGRAM CODE 4.7

```
int Fact(int n)
{
    if(n <= 1)
        return 1;
    else
        return (n * Dummy(n - 1));
}

void Dummy(int n)
{
    Fact(n);
}
```


Tail recursion –

- A recursive function is said to be tail recursive if there are no pending operations to be performed on return from a recursive call.
- Tail recursion is also used to return the value of the last recursive call as the value of the function.
- Tail recursion is advantageous as the amount of information that must be stored during computation is independent of the number of recursive calls.
- The Factorial() function is an example of a non-tail recursive function. The Binary_Search() function is an example of a tail recursive function.

Linear Recursion -

- Depending on the way the recursion grows, it is classified as linear or tree.
- A recursive function is said to be linearly recursive when no pending operation involves another recursive call, for example, the Fact() function.
- This is the simplest form of recursion and occurs when an action has a simple repetitive structure consisting of some basic steps followed by the action again.
- The Factorial() function is an example of linear recursion.

Tree Recursion –

- In a recursive function, if there is another recursive call in the set of operations to be completed after the recursion is over, this is called a tree recursion.
- Examples of tree recursive functions are the quick sort and merge sort algorithms, the FibSeries algorithm, and so on.
- The Fibonacci function FibSeries() is defined as

$$\begin{aligned}\text{FibSeries}(n) &= 0, && \text{if } n = 0 \\ &= 1, && \text{if } n = 1 \\ &= \text{FibSeries}(n - 1) + \text{FibSeries}(n - 2), && \text{otherwise}\end{aligned}$$

Let $n = 5$.

$$\text{FibSeries}(0) = 0$$

$$\text{FibSeries}(1) = 1$$

$$\text{FibSeries}(2) = \text{FibSeries}(0) + \text{FibSeries}(1) = 1$$

$$\text{FibSeries}(3) = \text{FibSeries}(1) + \text{FibSeries}(2) = 2$$

$$\text{FibSeries}(4) = \text{FibSeries}(2) + \text{FibSeries}(3) = 3$$

$$\text{FibSeries}(5) = \text{FibSeries}(3) + \text{FibSeries}(4) = 5$$

Figure 4.1 demonstrates this explanation for $n = 4$.

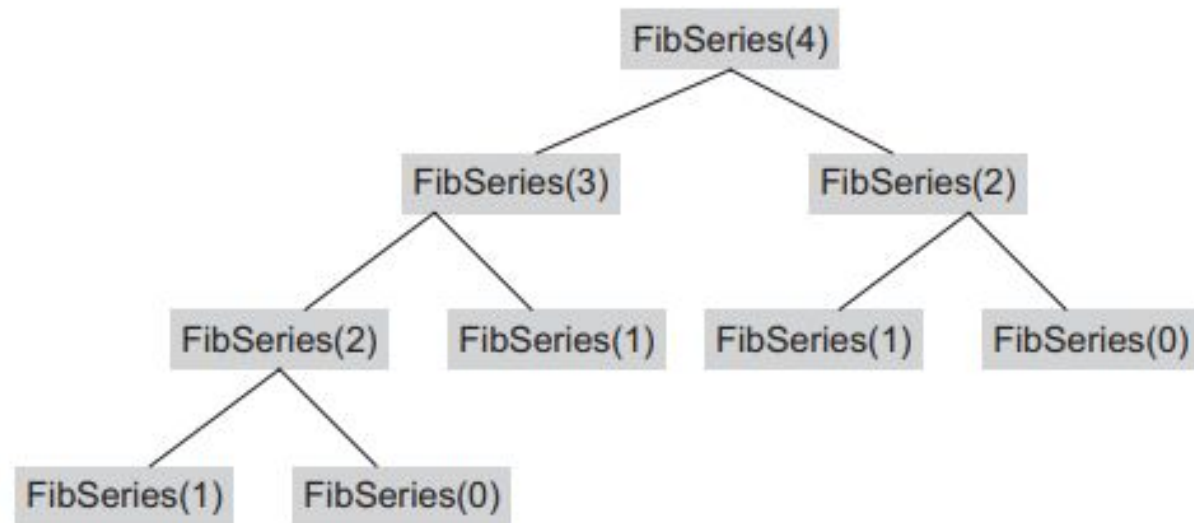
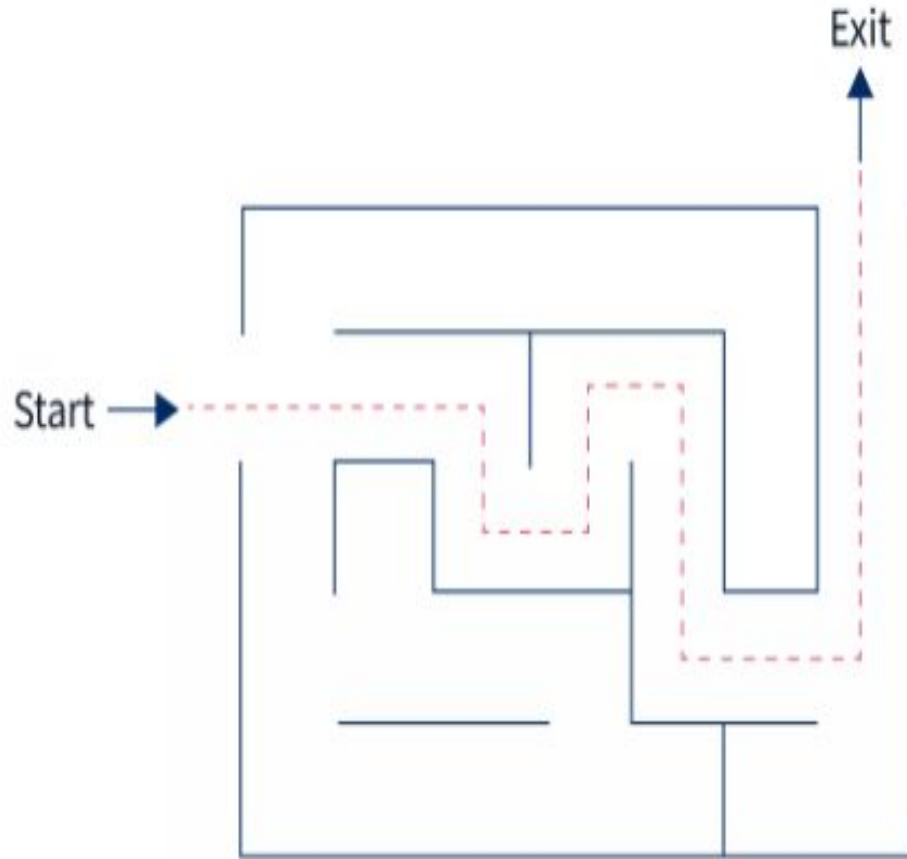


Fig. 4.1 Recursive calls in Fibonacci recursive function for $n = 4$

Backtracking algorithmic strategy

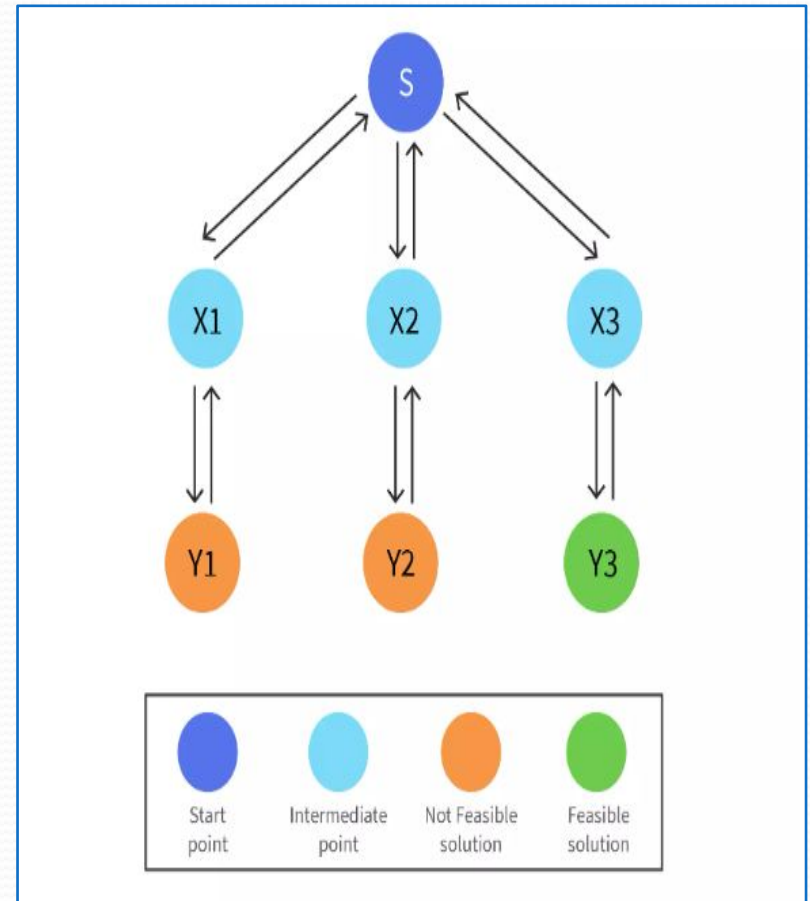
What is backtracking?



- Basically, the idea is --
 - we can build a solution step by step using recursion; e.g. In above example, we will try to find out multiple paths that can lead our desired exit door. And, we can travel into that path step by step using recursion.
 - If during the process we realise that is not going to be a valid solution, then we stop computing that solution and we return back to the step before (backtrack).
- Similarly, in programming, backtracking basically means that, if the current solution to your problem is not good or incorrect, you should eliminate it, move back (backtrack) and try another solution.
 - Backtracking searches all possible ways to solve a problem.
 - It basically does this by trying to find out all the possible solutions to a current problem and chooses the best solution among them.
 - It reaches to each possible solution using recursion.
 - Backtracking is used to solve a problem that have multiple solutions.

Backtracking algorithm

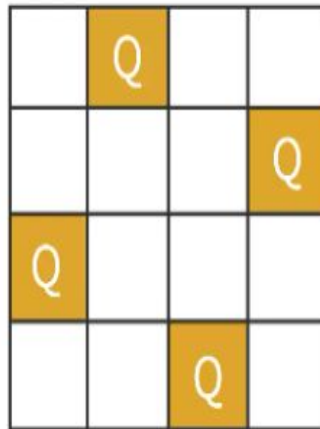
- For any backtracking problem, the backtracking algorithm tries to go through one of the paths to reach to the possible solution, and if the path doesn't lead them there, then the problem backtracks through the same path and takes another path in search of the solution.
- Consider the following space state tree. With the help of the below space state tree let's understand how the backtracking algorithm actually works.



Use of stack in backtracking -

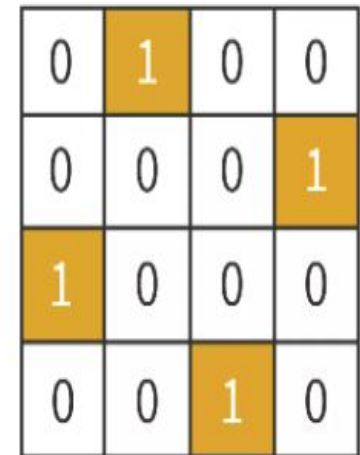
One of the algorithm design techniques is backtracking. For that aim, we dig into one path; if that path is inefficient, we return to the previous state and explore alternative options. We must save the previous state in order to return from the current state. Stack is required for this purpose. A famous example is N-queens problems.

Example of Backtracking Algorithm - We will discuss N Queen example which is a very popular problem that can be solved using Backtracking. In the N Queen problem, we have to place N queens on an $N \times N$ chessboard such that no two queens can attack each other. We cannot place two queens in the same row or same column or same diagonal in order to avoid the attack. For instance, given below is the configuration for placing N queens on $N \times N$ chessboard where $N=4$. In the output, we will print the binary matrix containing 1s where the queen can be placed, and the rest all cells will contain 0s.



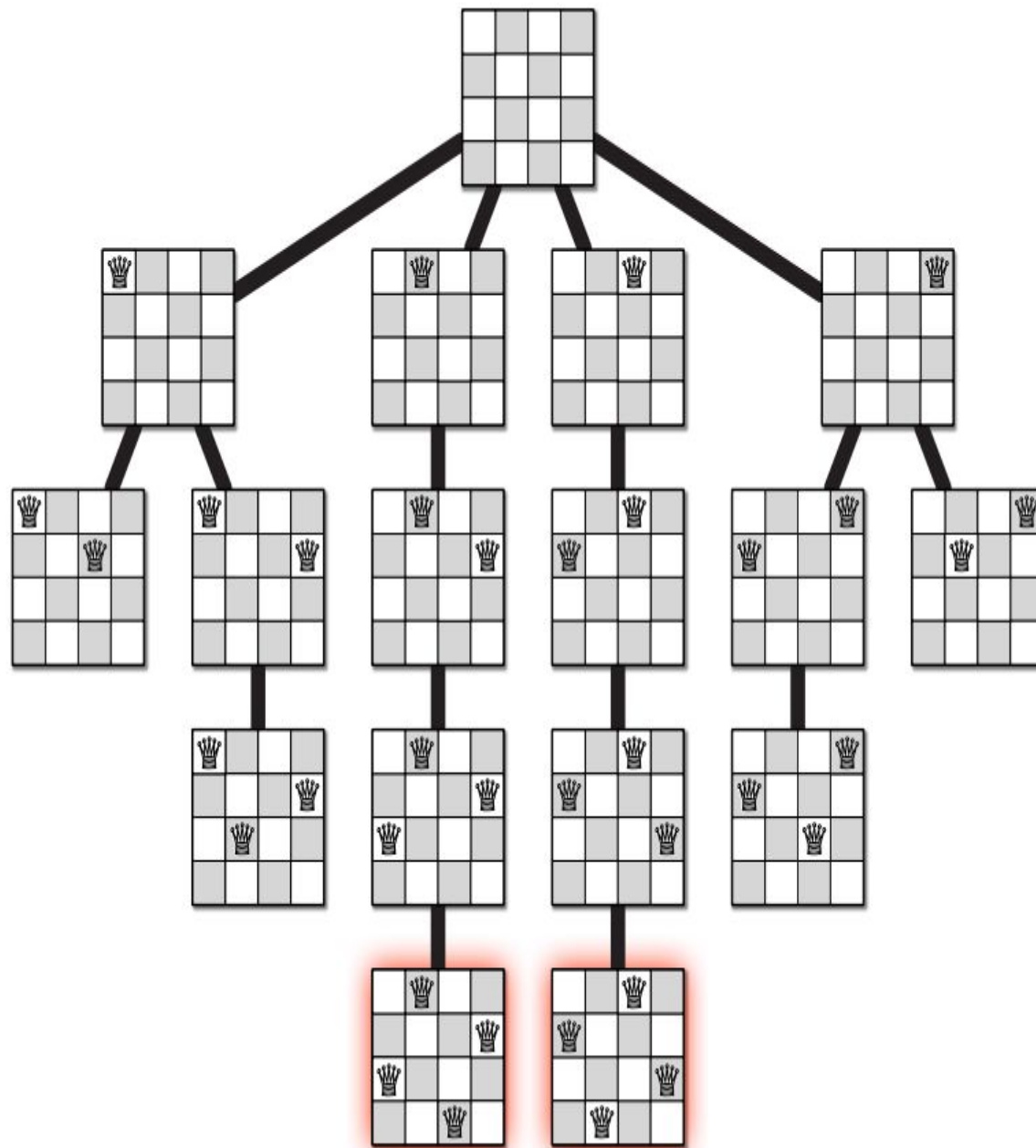
A 4x4 chessboard with four queens placed at (1,2), (2,4), (3,1), and (4,3) using 1-based indexing. The cells containing the queens are highlighted in yellow.

	Q		
			Q
Q			
		Q	



A 4x4 binary matrix representing the possible positions for queens. The cells with a value of 1 are highlighted in yellow, corresponding to the queen positions in the chessboard above.

0	1	0	0
0	0	0	1
1	0	0	0
0	0	1	0



- In the backtracking algorithm, we will start from the leftmost column and one by one try to place queens in different columns.
- When we place a queen in a column, we will check if the queen that we have placed currently is clashing with other already placed queens.
- In the current column, if we are able to find a row for which it is safe to place a queen then we will mark this row and column as part of the solution.
- If we do not find such a row due to clashes then we will backtrack and return false.
- **Approach to Code:** We will have a chessboard in the form of a 2d matrix in which we will try to place the queens. We will create the following three functions:
 1. **solveNQueen():** This will be the recursive function to solve n queen problem. It will take two arguments, a board matrix, and a column number.
 2. **isSafe_to_place():** This function will check if it is safe to place the queen in the current cell. It will take arguments board matrix, current row, and current column.
 3. **printOutput():** This function will print the output matrix.

```
/* C++ program to solve N Queen Problem */
```

```
#define N 4
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
/* A function to check if a queen can be safely placed on  
board[row][col]. */
```

```
bool isSafe_to_place(int board[N][N], int row, int col)  
{
```

```
    int i, j;
```

```
    /* Checking this row on left side */
```

```
    for (i = 0; i < col; i++)
```

```
        if (board[row][i])
```

```
            return false;
```

```
    /* Checking upper diagonal on left side */
```

```
    for (i = row, j = col; i >= 0 && j >= 0; i--, j--)
```

```
        if (board[i][j])
```

```
            return false;
```

```
    /* Checking lower diagonal on left side */
```

```
    for (i = row, j = col; j >= 0 && i < N; i++, j--)
```

```
        if (board[i][j])
```

```
            return false;
```

```
    return true;
```

```
}
```

/ A recursive function to solve N Queen problem */*

```
bool solveNQueen(int board[N][N], int col)
```

```
{
```

/ base case: If all queens are placed then return true */*

```
    if (col >= N)
```

```
        return true;
```

/ Consider this column and try placing this queen in all rows one by one */*

```
    for (int i = 0; i < N; i++) {
```

/ Check if the queen can be placed on board[i][col] */*

```
    if (isSafe_to_place(board, i, col)) {
```

/ Place this queen in board[i][col] */*

```
        board[i][col] = 1;
```

/ recur to place rest of the queens */*

```
        if (solveNQueen(board, col + 1))
```

```
            return true;
```

/ If queen cannot be placed then simply backtrack and try other configurations */*

```
        board[i][col] = 0; // BACKTRACK
```

```
    }
```

```
}
```

/ If the queen cannot be placed in any row in the current column then return false */*

```
    return false;
```

```
}
```

```
/* function to print output solution */
```

```
void printOutput(int board[N][N])  
{  
    for (int i = 0; i < N; i++) {  
        for (int j = 0; j < N; j++)  
            cout<<board[i][j];  
        cout<<"\n";  
    }  
}
```

```
/* This function solves the N Queen problem using  
Backtracking. */
```

```
bool solveNQ()  
{  
    int board[N][N] = { { 0, 0, 0, 0 },  
                          { 0, 0, 0, 0 },  
                          { 0, 0, 0, 0 },  
                          { 0, 0, 0, 0 } };  
  
    if (solveNQueen(board, 0) == false) {  
        cout<<"Solution does not exist";  
        return false;  
    }  
    printOutput(board);  
    return true  
}
```

```
// driver  
program to test  
above function
```

```
int main()  
{  
    solveNQ();  
    return 0;  
}
```